جزوهٔ درسی · جلسهٔ ۱ از ۵

چرخهٔ جدید تولید نرم‌افزار با کدنویسی شهودی

از پرامپت‌های (prompt) نامنظم تا مهندسی عامل‌محور (Agentic engineering)

مطالعهٔ پیشنهادی: ۴۵ دقیقه **وابستگی** (dependency)**:** پیش‌نیاز چهار جلسهٔ بعدی

فهرست مطالب

1. مقدمه: تغییری که از جنس ابزار نیست
2. عامل چیست؟ مروری کوتاه اما دقیق
3. طیف: از کدنویسی شهودی تا مهندسی عامل‌محور
4. مهندسی زمینه: مهارت واقعی
5. چرخهٔ جدید تولید نرم‌افزار
6. مهندسی مهار: آنچه مدل را احاطه می‌کند
7. نقش در حال تحول توسعه‌دهنده
8. عامل‌های کدنویسی در عمل
9. هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی
10. از کجا شروع کنیم
11. جمع‌بندی و سه اصل ماندگار
12. خودآزمایی

۱. مقدمه: تغییری که از جنس ابزار نیست

در بیشتر تاریخ رایانش، برنامه‌نویسی در اصل نوعی «ترجمه» بوده است: مسئله را به زبان انسانی بفهم، راه‌حل را به زبان انتزاعی طراحی کن، و بعد آن را با نحوی بازنویسی کن که ماشین بتواند اجرایش کند. هر کدام از این گام‌ها دردسر و کندی خودش را دارد، و این موانع حالا دارند از میان می‌روند.

> بزرگ‌ترین تحول در مهندسی نرم‌افزار، یک زبان، چارچوب یا سرویس ابری تازه نیست؛ گذار از **نوشتن کد** به **بیان قصد** (intent) است، و سپردن این کار به سامانه‌های هوشمند که آن قصد را به نرم‌افزاری کارآمد تبدیل کنند.

این تحول یک‌شبه رخ نداد. با تکمیل خودکار شروع شد، یعنی پیش‌بینی سادهٔ توکن بعدی در ویرایشگر. سپس پیشنهادهای درون‌خطی آمد که می‌توانستند یک تابع کامل را بنویسند. بعد رابط‌های گفت‌وگومحور اجازه دادند ویژگی‌ها را به زبان طبیعی توصیف کنیم و پیاده‌سازی بگیریم. و اکنون عامل‌های (agent) کاملاً خودگردان (autonomous) ریپوها (repository) را کلون می‌کنند، تغییرهایی را که چند فایل را درگیر می‌کنند برنامه‌ریزی و اجرا می‌کنند، در محیط‌های سندباکسی تست (test) می‌گیرند و پول‌ریکوئست (pull request) می‌سازند — بدون آنکه انسانی حتی یک خط کد تایپ کند.

اعداد وضعیت موجود (اوایل ۲۰۲۶)

۸۵٪ توسعه‌دهندگان حرفه‌ای مرتب از عامل‌های کدنویسی استفاده می‌کنند، ۵۱٪ روزانه، و برآورد می‌شود ۴۱٪ از کدهای تازه‌نوشته‌شده تولید هوش مصنوعی باشد.

اما این دگرگونی یکنواخت یا ساده نیست. طیفی وجود دارد که یک سرِ آن «کدنویسی شهودی (Vibe coding)» است — توسعه‌دهنده پرامپتی (prompt) می‌دهد و هرچه برگشت را می‌پذیرد — و سر دیگرش «مهندسی عامل‌محور»، جایی که هوش مصنوعی موتور نیرومند پیاده‌سازی است، اما درون سامانه‌ای از قیدها، تست‌ها و حلقه‌های (loop) بازخوردِ آگاهانه‌طراحی‌شده کار می‌کند و انسان نظارت بر معماری، درستی و کیفیت را نگه می‌دارد.

چرا این تمایز، تمایزی مدیریتی هم هست

اگر به مدیر ارشد فناوری بگویید تیم شما سامانهٔ پردازش پرداخت را «شهودی» کدنویسی می‌کند، باید زنگ خطر به صدا دربیاید — و درست هم هست. اگر به همان فرد بگویید تیم «مهندسی عامل‌محور» می‌کند و هوش مصنوعی زیر قیدهای انسانی‌طراحی‌شده پیاده‌سازی می‌کند و پوشش (coverage) تست درستی را تضمین می‌کند، گفت‌وگو از بنیاد فرق می‌کند.

۲. عامل چیست؟ مروری کوتاه اما دقیق

پیش از هر چیز به تصویری مشترک نیاز داریم. **عامل هوش مصنوعی** سامانه‌ای نرم‌افزاری است که یک هدف را درک می‌کند، گام‌های رسیدن به آن را برنامه‌ریزی می‌کند، از راه ابزارها کنش انجام می‌دهد، نتیجه را مشاهده می‌کند و آن‌قدر تکرار می‌کند تا به هدف برسد یا به شرط توقف بخورد.

تفاوت بنیادی با ربات گفت‌وگو در همین‌جاست: ربات گفت‌وگو پاسخی تولید می‌کند و منتظر پرامپت بعدی می‌ماند؛ عامل حلقهٔ خودش را می‌چرخاند. شما هدف را در بالا می‌گذارید و او در هر گام تصمیم می‌گیرد قدم بعدی چیست.

حلقهٔ عامل

ادراک ← برنامه ← کنش ← مشاهده ← تکرار. این حلقه، قلب تپندهٔ هر عامل است. هر چیزی که در ادامهٔ این دوره می‌بینید — از ابزارها و مهارت‌ها (skill) تا ارزیابی (evaluation) و امنیت — صورتی از همین حلقه یا مراقبتی بر آن است.

پنج جزء سازنده

جزء	نقش	اگر نباشد چه می‌شود
مدل	موتور استدلال؛ زمینهٔ جاری را می‌خواند و تصمیم می‌گیرد گام بعد چه باشد: تفکر بعدی، فراخوان ابزار (tool call) بعدی، یا پیام بعدی.	سامانه فقط یک اسکریپت ثابت است.
ابزارها	اتصال مدل به جهان: APIهایی که می‌تواند فراخوانی کند، کدی که اجرا می‌کند، پایگاه‌داده‌ای که پرس‌وجو می‌کند، عامل دیگری که کار را به او می‌سپارد.	عامل فقط حرف می‌زند، کاری نمی‌کند.
حافظه (memory)	وضعیت؛ یادآوری تعامل‌های گذشته، بازیابی (retrieval) قواعد پروژه، و نگه‌داشتن زمینه (context) میان نشست‌ها (session) تا هرگز از صفحهٔ سفید شروع نکند.	هر نشست از صفر شروع می‌شود و همان اشتباه‌ها تکرار می‌شوند.
ارکستراسیون (orchestration)	کدی که حلقه را می‌چرخاند: زمینهٔ هر فراخوان را می‌سازد، فراخوان ابزار (tool) را می‌فرستد، نتیجه را می‌گیرد و تصمیم می‌گیرد ادامه بدهد یا نه.	حلقه‌ای در کار نیست؛ فقط یک پرسش و پاسخ.
دیپلوی (deploy)	آنچه نمونهٔ اولیه را به سرویس تبدیل می‌کند: میزبانی، هویت، رصدپذیری (observability) و زیرساخت تولید.	نمونهٔ اولیه روی لپ‌تاپ باقی می‌ماند.

۳. طیف: از کدنویسی شهودی تا مهندسی عامل‌محور

خاستگاه واژه

در فوریهٔ ۲۰۲۵، آندری کارپثی شیوه‌ای از برنامه‌نویسی را توصیف کرد که به‌سرعت در جامعهٔ مهندسی نرم‌افزار مورد استقبال و استفاده قرار گرفت: «کاملاً تسلیم حس‌وحال شو، رشد شتابان توانایی‌های هوش مصنوعی را بپذیر و فراموش کن که اصلاً کدی وجود دارد.» در این حالت، توسعه‌دهنده خواسته‌اش را به زبان طبیعی می‌گوید، خروجی را می‌پذیرد و وقتی چیزی خراب شد، پیام خطا را در پرامپت بعدی کپی می‌کند و می‌گوید درستش کن.

این اصطلاح فراگیر شد چون چیز واقعی‌ای را نام‌گذاری کرد: بسیاری از توسعه‌دهندگان از قبل همین‌طور کار می‌کردند اما واژه‌ای برایش نداشتند. اما ظرف چند ماه، «کدنویسی شهودی» به هر گردش‌کارِ مبتنی بر هوش مصنوعی اطلاق شد و همین، معنایش را رقیق کرد. تا اوایل ۲۰۲۶ خودِ کارپثی پذیرفت که تعریف اولیه‌اش بیش از حد محدود بوده است و اصطلاح **مهندسی عامل‌محور** را برای سرِ ساختارمند این طیف پیشنهاد کرد.

طیف، نه دوگانه

مفیدتر است این دو را نه به‌صورت یک دو انتخابی، بلکه به‌صورت دو سرِ یک طیف ببینیم. تمایزدهنده این نیست که از هوش مصنوعی استفاده می‌کنید یا نه؛ این است که **چه میزان ساختار، راستی‌آزمایی و قضاوت انسانی** خروجی آن را احاطه کرده است.

بُعد	کدنویسی شهودی	کمک‌گرفتن ساختارمند	مهندسی عامل‌محور
بیان قصد	پرامپت‌های گفتاری و سرِدستی	پرامپت‌های دقیق همراه با مثال و قید (constraint)	مشخصات (spec) رسمی، سند معماری، فایل‌های حافظه
راستی‌آزمایی	«به‌نظر کار می‌کند؟»	تست دستی، وارسی نمونه‌ای	مجموعه‌تست خودکار، دروازه‌های CI/CD (gate)، داوران مدل‌زبانی
فهم کدبیس	حداقلی؛ ممکن است کد تولیدشده اصلاً خوانده نشود	بازبینی گزینشی مسیرهای حساس	بازبینی جامع معماری؛ جزئیات پیاده‌سازی با عامل
مدیریت خطا	کپی پیام خطا و بازگرداندن به مدل	انسان علت ریشه‌ای را تشخیص می‌دهد، عامل اصلاح را پیاده می‌کند	عامل درون مرزهای تعریف‌شده خودتشخیص می‌دهد؛ مسائل معماری با انسان
دامنهٔ مناسب	نمونهٔ اولیه، اسکریپت، پروژهٔ شخصی، هکاتون	افزودن ویژگی به کدبیس‌های موجود	سامانه‌های تولیدی، توسعه در مقیاس تیم
نمای ریسک	بالا؛ برای کد دورریختنی (Disposable code) پذیرفتنی	متوسط؛ قضاوت انسانی در ایستگاه‌های کلیدی	پایین؛ راستی‌آزمایی نظام‌مند در هر مرحله

جدول ۱ — طیف از کدنویسی شهودی تا مهندسی عامل‌محور

> **نکتهٔ کاربردی**
>
> جای درست شما روی این طیف را **میزان مخاطره** تعیین می‌کند. یک نمونهٔ اولیهٔ آخر هفته می‌تواند کاملاً شهودی باشد. یک API تولیدی که تراکنش مالی را پردازش می‌کند، مهندسی عامل‌محور می‌طلبد. بیشتر کارهای واقعی جایی در میانه‌اند و مهارت، در دانستن این است که برای هر کار خط را کجا بکشید.

راستی‌آزمایی: تمایزدهندهٔ اصلی

بزرگ‌ترین تفاوت دو سر طیف در نحوهٔ راستی‌آزمایی خروجی‌هاست. در کدنویسی شهودی، راستی‌آزمایی اختیاری است؛ کد را اجرا می‌کنی و می‌بینی درست به‌نظر می‌رسد یا نه. در مهندسی عامل‌محور دو سازوکار با هم کار می‌کنند:

- **تست‌ها** بخش قطعی (deterministic) سامانه را می‌سنجند: تابعی با این ورودی باید آن خروجی را بدهد. داورشان کد است.
- **ارزیابی‌ها** بخش غیرقطعی (not deterministic) را می‌سنجند: آیا عامل مسیر (trajectory) درستی از گام‌ها را طی کرد؟ ابزار درست را انتخاب کرد؟ پاسخ نهایی به حد کیفی رسید؟ داورشان مجموعه‌دادهٔ برچسب‌خورده، سنجه‌های (rubric) نمره‌دهی و داوران مدل‌زبانی‌اند.

> **قاعدهٔ سخت**
>
> بدون هر دو، کاری که انجام می‌دهید کدنویسی شهودی است — مهم نیست پرامپت‌هایتان چقدر پیچیده و حرفه‌ای باشند.

۴. مهندسی زمینه: مهارت واقعی

با بلوغ این حوزه یک بینش کلیدی پدیدار شد: کیفیت کد تولیدشده کمتر به زیرکی پرامپت‌ها و بیشتر به کیفیت **زمینه**‌ای بستگی دارد که فراهم کرده‌اید. این همان چیزی است که **مهندسی زمینه** (Context engineering) نام گرفته: تأمین اطلاعات غنی و ساختارمند دربارهٔ کدبیس، معماری، قراردادها و قصد شما.

شش نوع زمینه

1. **دستورها** (Instructions): نقش اصلی عامل، هدف‌ها و مرزهای عملیاتی‌اش.
2. **دانش** (Knowledge): سندهای بازیابی‌شده، نمودارهای معماری و دادهٔ تخصصی حوزه.
3. **حافظه** (Memory): گزارش‌های کوتاه‌مدت نشست (چه گذشت) و وضعیت ماندگار بلندمدت (این پروژه چیست).
4. **مثال‌ها** (Examples): نمونه‌های رفتاری چندشاتی و الگوهای مرجع درون کدبیس.
5. **ابزارها** (Tools): تعریف دقیق APIها، اسکریپت‌ها و سرویس‌های بیرونی قابل فراخوانی.
6. **گاردریل‌ها** (guardrail): قیدهای سخت، قواعد قالب‌بندی و اعتبارسنجی‌های ایمنی.

زمینهٔ ایستا در برابر زمینهٔ پویا

زمینهٔ ایستا (Static) همیشه بارگذاری می‌شود: سیستم پرامپت‌ها، فایل‌های قاعده (`AGENTS.md`، `CLAUDE.md`، `GEMINI.md`)، حافظهٔ سراسری و تعریف شخصیت. این زمینه می‌گوید عامل کیست و چطور رفتار می‌کند. گران است، چون هر توکنش در هر تعامل حاضر است، ربطی داشته باشد یا نه.

زمینهٔ پویا (dynamic) هنگام نیاز بارگذاری می‌شود: دستورهای مهارتی که با کار جاری هم‌خوان است، نتایج ابزارها، سندهای بازیابی‌شده و بخش اخیر تاریخچهٔ نشست. کارآمد است، چون فقط وقتی هزینهٔ توکن (token) می‌پردازید که اطلاعات لازم باشد.

> **این یک موازنهٔ مهندسی واقعی است**
>
> زمینهٔ ایستای بیش‌ازحد زیاد، توکن هدر می‌دهد و سیگنال را رقیق می‌کند. زمینهٔ ایستای کم‌ازحد یعنی عامل قواعد حیاتی را فراموش می‌کند. سامانه‌های خوب طراحی شده، این مرز را یک **تصمیم معماری درجه‌یک** می‌دانند: بازبینی‌شده و نسخه‌بندی‌شده، مثل هر پیکربندی دیگر.

مهارت‌های عامل: قوی‌ترین الگوی زمینهٔ پویا

مهارت‌های عامل بسته‌های ساختارمند و قابل‌حملی از دانش رویه‌ای (procedural knowledge) هستند که عامل فقط وقتی آن‌ها را بارگذاری می‌کند که کار اقتضا کند. به‌جای جاسازی هر دانش تخصصی در سیستم پرامپت، عامل همچنان عمومی و سبک می‌ماند که به‌اقتضا در نقش متخصص فرو می‌رود — سازوکاری به نام **افشای تدریجی** (Progressive disclosure):

1. در آغاز، عامل فقط فراداده‌های سبک هر مهارت را می‌بیند.
2. وقتی کاری با یک مهارت تطبیق پیدا کرد، دستور (instruction) کامل آن بارگذاری می‌شود.
3. مرجع تفصیلی فقط در صورت نیاز صریح خوانده می‌شود.

نتیجه این است که عامل می‌تواند ده‌ها توانایی تخصصی حمل کند و فقط برای همانی که همین حالا فعال است توکن بپردازد. مهارت‌ها به‌سرعت در عامل‌های کدنویسی و سکوهای سازمانی پذیرفته شدند، چون چهار مشکل دیرینه را حل می‌کنند: پوسیدگی زمینه (Context rot) بر اثر پرامپت‌هایی که بیش از حد در آن‌ها مطلب گنجانده شده، نبود حافظهٔ رویه‌ای (Procedural memory) در مدل‌های زبانی، سربار عملیاتی معماری‌های چندعاملی، و نیاز به قابلیت حمل میان ابزارها و ارائه‌دهندگان.

> **ادامهٔ این بحث**
>
> جلسهٔ سوم دوره کاملاً به همین موضوع اختصاص دارد: طراحی و مدیریت نشست، نوشتن و ارزیابی مهارت، ساخت حافظهٔ ماندگار، و بهینه‌سازی اقتصاد توکن برای سامانه‌های تولیدی.

> پرسش درست این نیست که «چطور هوش مصنوعی را فریب بدهم تا کد خوب بنویسد؟» پرسش درست این است: «یک هم‌تیمی تازه‌وارد چه چیزی باید بداند تا بتواند مشارکت مؤثر داشته باشد، و چطور آن دانش را به شکلی در بیاورم که هوش مصنوعی هم بتواند به کارش ببرد؟»

۵. چرخهٔ جدید تولید نرم‌افزار

چرخهٔ سنتی زیر فشار

چرخهٔ تولید نرم‌افزار پیش‌تر هم یک دگرگونی بزرگ را از سر گذرانده است: گذار از فرایندهای آبشاری به مدل‌های تکرارشونده — دوره‌های چابک، سی‌آی، خطوط لولهٔ DevOps و انتشار سریع. آن تغییر، حلقه‌های بازخورد را کوتاه کرد و دیپلوی را از رویدادی فصلی به فرایندی پیوسته بدل ساخت.

هوش مصنوعی این چرخه را به‌شدت فشرده می‌کند، اما **نامتوازن**: پیاده‌سازی‌ای که هفته‌ها می‌برد اکنون ساعت‌ها طول می‌کشد، در حالی که نیازمندی، معماری و راستی‌آزمایی همچنان با سرعت انسان پیش می‌روند. نتیجه، نسخهٔ سریع‌ترِ چرخهٔ قدیمی نیست؛ گردش‌کاری متفاوت است که در آن مرز فازها محو می‌شود، تکرارها از هفته به دقیقه می‌رسد و توسعه‌دهنده از پیاده‌سازِ اصلی به طراح سامانه و داور (judge) کیفیت بدل می‌شود.

> **دربارهٔ سرعت تغییر**
> تصویر فازبه‌فازِ زیر، وضعیت میانهٔ ۲۰۲۶ است و به‌سرعت در حال جابه‌جایی است. نشانه‌ها می‌گویند فشرده‌سازی از پیاده‌سازی فراتر می‌رود: تیم‌ها همین حالا گردش‌کارهایی را می‌آزمایند که توسعه‌دهنده مستقیم از مشخصات به بازبینی می‌رود و عامل‌ها پیاده‌سازی، تست و دیپلوی را در پس‌زمینه انجام می‌دهند. آنچه ثابت می‌ماند، قضاوت انسانی، ذائقه و مهارت راستی‌آزمایی است.

تحول هر فاز

نیازمندی و برنامه‌ریزی

نیازمندی‌ها مرحله‌ای است که همیشه بیشترین فاصله را میان آنچه واقعاً خواسته شده و آنچه در عمل ساخته می‌شود داشته است. ابزارهای امروزی مستقیماً در پالایش نیازمندی مشارکت می‌کنند: تولید داستان کاربر از خلاصهٔ محصول، شناسایی حالت‌های مرزی‌ای که انسان از قلم می‌اندازد، ساخت اسکیمای API از توصیف زبان طبیعی، و ساخت نمونهٔ اولیهٔ تعاملی از سند مشخصات. **نیازمندی دیگر سندی نیست که میان تیم‌ها دست‌به‌دست شود؛ گفت‌وگویی است که هم‌زمان مشخصات و پیاده‌سازی اولیه تولید می‌کند.**

طراحی و معماری

معماری سرسخت‌ترین فاز در برابر خودکارسازی است، و دلیل خوبی هم دارد: تصمیم‌های معماری بنیاداً دربارهٔ موازنه‌اند — سازگاری در برابر دسترس‌پذیری، پیچیدگی در برابر انعطاف، ساخت در برابر خرید — و این موازنه‌ها به زمینهٔ کسب‌وکار، قیدهای سازمانی و ملاحظات راهبردی بلندمدت وابسته‌اند. هوش مصنوعی در **اجرای** تصمیم معماری عالی است: با یک سند معماری روشن، می‌تواند کل اسکلت برنامه را بسازد، الگوها را در ماژول‌ها یکدست کند و کد تازه را با قراردادهای موجود هماهنگ نگه دارد.

پیاده‌سازی

عامل‌های امروزی می‌توانند یک ویژگی کامل را از توصیف زبان طبیعی بسازند و تغییرهایی سازگار تولید کنند که چند فایل را درگیر می‌کنند. سود بهره‌وری واقعی است: پیمایش‌های صنعتی ۲۵ تا ۳۹ درصد بهبود گزارش می‌کنند.

> **تصویر پیچیده‌تر از تیترها**
>
> پژوهش METR نشان داد توسعه‌دهندگان باتجربه در برخی کارها با دستیار هوش مصنوعی ۱۹٪ **کندتر** شدند؛ عمدتاً به‌سبب زمانی که صرف راستی‌آزمایی، دیباگ (debug) و اصلاح خروجی می‌شد. هوش مصنوعی کار پیاده‌سازی را حذف نمی‌کند؛ آن را از نوشتن به بازبینی، هدایت و راستی‌آزمایی تبدیل می‌کند.

تست و تضمین کیفیت

آزمودن کد تولیدشده با هوش مصنوعی یعنی سنجش نه‌فقط آنچه عامل تولید کرد، بلکه **مسیری که طی کرد**. ارزیابی خروجی، مصنوع نهایی را می‌سنجد (کامپایل می‌شود؟ تست‌ها سبزند؟) و ارزیابی مسیر، کل توالی فراخوان ابزارها و استدلال میانی را. هر دو لازم‌اند، چون خروجی روانی که گام‌های راستی‌آزمایی‌اش را میان‌بر زده، از خروجی با خطای آشکار خطرناک‌تر است.

هوش مصنوعی خودِ تولید تست را هم دگرگون می‌کند: عامل‌ها می‌توانند حالت‌های مرزی و تست‌های مبتنی بر ویژگی تولید کنند که به ذهن انسان نمی‌رسد. مهم‌تر از آن، تست‌ها و ارزیابی‌ها به سازوکار اصلی **انتقال قصد** به عامل بدل می‌شوند: یک مجموعهٔ ارزیابی خوب‌نوشته، دقیق‌تر از هر پرامپت زبان طبیعی می‌گوید «درست» یعنی چه.

> **چرخهٔ پیوستهٔ بهبود کیفیت**
>
> ارزیابی در برابر مجموعهٔ بنچمارک (benchmark) ← خوشه‌بندی علل خطا ← بهینه‌سازی پرامپت‌ها یا ابزارهایی که باعثش بودند ← راستی‌آزمایی اصلاح در برابر مجموعهٔ بازگشتی ← پایش ترافیک تولید برای یافتن حالت‌های شکست (failure modes) تازه. هر دور، دور بعدی را تقویت می‌کند.

بازبینی کد و دیپلوی

هوش مصنوعی به‌عنوان بازبینِ نوبت اول عمل می‌کند: یافتن اشکال‌های محتمل، نقض سبک، آسیب‌پذیری‌های امنیتی و مسائل کارایی پیش از آنکه بازبین انسانی کد را ببیند. این جایگزین بازبینی انسانی نیست — تصمیم‌های وابسته به زمینه دربارهٔ طراحی، نگه‌داشت‌پذیری و هم‌راستایی راهبردی هنوز قضاوت انسانی می‌خواهند — اما بار شناختی بازبین را به‌شکل معناداری کم می‌کند. خطوط لولهٔ دیپلوی هم آگاه‌تر می‌شوند: پایش سلامت انتشار، بازگرداندن خودکار انتشار مشکل‌دار و پیش‌بینی ریسک بر پایهٔ ماهیت و دامنهٔ تغییر.

نگه‌داری و تکامل

شاید بزرگ‌ترین تغییری که کمتر به آن توجه شده، همین‌جا باشد. کدبیس‌های قدیمی‌ای که سر درآوردن از آن‌ها برای تازه‌واردان تقریباً ناممکن بود، اکنون قابل پیمایش، فهم و تغییرند. این تغییر اثر بزرگی بر بدهی فنی دارد: کدی که چون فقط نویسندگان اصلی‌اش از آن سر درمی‌آوردند، کسی جرئت دست‌زدن به آن را نداشت، حالا می‌تواند با اطمینان بازسازی و نوسازی شود. مهاجرت میان چارچوب‌ها، به‌روزرسانی APIهای منسوخ و نوسازی مجموعه‌تست‌ها — کارهایی که پیش‌تر آن‌قدر خسته‌کننده و پرریسک بودند که عملاً هرگز انجام نمی‌شدند.

مدل کارخانه

همهٔ این تغییرها را می‌توان در قالب یک مدل ذهنی کنار هم دید: **مدل کارخانه** (Factory model). در این مدل، محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه **سامانه‌ای است که کد را تولید می‌کند**. این سامانه شامل مشخصات و زمینه‌ای است که تعریف می‌کند چه باید ساخته شود؛ عامل‌هایی که مشخصات را به پیاده‌سازی ترجمه می‌کنند؛ تست‌ها و دروازه‌های کیفیت که درستیِ کار را می‌سنجند؛ حلقه‌های بازخوردی که خطاها را برای اصلاح به عامل برمی‌گردانند؛ و گاردریل‌هایی که رفتار عامل را ایمن و قابل‌پیش‌بینی نگه می‌دارند.

> مدیر کارخانه قطعه‌ها را خودش با دست سرِهم نمی‌کند؛ خط مونتاژ را طراحی می‌کند و از کیفیت محصول اطمینان می‌یابد. توسعه‌دهندهٔ امروزی هم سامانهٔ توسعه را طراحی می‌کند و مطمئن می‌شود خروجی آن به استاندارد لازم می‌رسد.

۶. مهندسی مهار: آنچه مدل را احاطه می‌کند

کسانی که تازه ساختن عامل را شروع می‌کنند، وسوسه می‌شوند مدل را همان کل سامانه بدانند: مدل تازه‌ای می‌آید و عامل باهوش‌تر می‌شود؛ مدل قدیمی‌تر باشد، عامل ضعیف‌تر می‌شود. در این نگاه، هر نقطهٔ قوت یا ضعفی به مدل نسبت داده می‌شود. این برداشت غلط است و به سرمایه‌گذاری‌های اشتباه می‌انجامد.

معادلهٔ مفید

عامل = مدل + مهار (harness). مدل خام به‌تنهایی عامل نیست؛ وقتی به عامل تبدیل می‌شود که مهار، وضعیت، امکان اجرای ابزار، حلقهٔ بازخورد و قیدهای الزام‌آور را در اختیارش بگذارد. رفتاری که هنگام کار با Claude Code، Cursor، Codex، Antigravity، Aider یا Cline تجربه می‌کنید، بیش از آنکه محصول مدل زیرین باشد، محصول کاری است که مهار انجام می‌دهد.

داخل مهار چیست؟

- **دستورها و فایل‌های قاعده:** متنی که می‌گوید عامل کیست، چه چیزی برایش اهمیت دارد و چه کارهایی برایش ممنوع است؛ از جمله `AGENTS.md` ، `CLAUDE.md` ، `GEMINI.md` ، فایل‌های مهارت و پرامپت‌های زیرعامل‌ها (subagent).
- **ابزارها:** توابع، سرورهای MCP و APIهایی که عامل می‌تواند فراخوانی کند، به‌همراه توضیحاتی که به مدل می‌گوید کِی و چگونه از آن‌ها استفاده کند.
- **سندباکس** (sandbox) **و محیط‌های اجرا:** کد عامل واقعاً کجا اجرا می‌شود، به چه چیزهایی دسترسی دارد و به چه چیزهایی ندارد.
- **منطق ارکستراسیون:** راه‌اندازی زیرعامل‌ها، مسیریابی (routing) میان مدل‌ها، واگذاری کار میان عامل‌های متخصص و قواعدی که تعیین می‌کنند هرکدام کِی فعال شوند.
- **گاردریل‌ها یا قلاب‌ها** (hook)**:** کد قطعی که در نقاط مشخصی از چرخهٔ عمر اجرا می‌شود: پیش از فراخوان ابزار، پس از ویرایش فایل، پیش از کامیت (commit). قلاب‌ها برای کارهایی‌اند که عامل هرگز نباید فراموش کند، اما اغلب فراموش می‌کند.
- **رصدپذیری:** لاگ‌ها، ردیابی‌ها، ارزیابی‌ها، سنجش هزینه و تأخیر (latency). بدون رصدپذیری، هیچ راهی نیست بفهمیم عامل خوب کار می‌کند یا بی‌صدا از مسیر خارج شده است.

اگر این محدوده گسترده به‌نظر می‌رسد، واقعاً هم گسترده است؛ و مهم‌تر اینکه **مسئولیت آن با تیم شماست**، نه ارائه‌دهندهٔ مدل.

مهار در فازهای چرخهٔ عمر

فاز	اجزای فعال مهار	کاری که انجام می‌شود
۱. نیازمندی، برنامه و معماری پیکربندی مهار	دستورها و فایل‌های قاعده	پیش از آنکه هوش مصنوعی کد تولیدی بنویسد، توسعه‌دهنده محیط عامل را برپا می‌کند: ساختن `AGENTS.md`، تعریف قیدهای معماری، مشخص‌کردن ابزارهای در دسترس و قواعدی که عامل حق شکستن‌شان را ندارد.
۲. پیاده‌سازی اجرای مهار	سندباکس، محیط اجرا، ابزارها	مدل کد تولید می‌کند و آن را درون سندباکس ایزولهٔ مهار اجرا می‌کند. خواندن فایل یا جست‌وجوی وب فقط از راه ابزارهایی ممکن است که مهار فراهم کرده.
۳. تست و تضمین کیفیت حلقهٔ بازخورد	منطق ارکستراسیون، گاردریل‌ها	مهار محیط اجرای تست را فراهم می‌کند؛ اگر تستی شکست بخورد، منطق ارکستراسیون خروجی خطا را می‌گیرد و به مدل برمی‌گرداند و می‌گوید دوباره تلاش کن. همین، حلقهٔ خودکار «بیندیش ← کنش کن ← مشاهده کن» را می‌سازد.
۴. بازبینی، دیپلوی و نگه‌داری رصد مهار	قلاب‌ها، رصدپذیری	قلاب قطعی جلوی کامیتی را می‌گیرد که گذرواژهٔ سخت‌کدشده دارد. لایهٔ رصدپذیری هزینهٔ توکن، تأخیر و انحراف `(drift)` عامل را دنبال می‌کند تا مهندسان بتوانند ممیزی کنند که عامل چرا تصمیم خاصی گرفت.

اثر مهار قابل اندازه‌گیری است

بنچمارک‌های عمومی اندازهٔ این اثر را ملموس می‌کنند. در `Terminal Bench 2.0` یک تیم فقط با تغییر مهار — بدون هیچ تغییری در مدل — عاملی را از بیرونِ سی عامل برتر به جمع پنج عامل برتر رساند. پژوهشی جداگانه در `LangChain` با دست‌کاری تنها سیستم پرامپت، ابزارها و میان‌افزار پیرامون مدلی ثابت، امتیاز همان بنچمارک را ۱۳٫۷ واحد بالا برد.

گذار از کدنویسی شهودی به مهندسی عامل‌محور صرفاً دربارهٔ ابزاری که به کار می‌برید نیست — یک توسعه‌دهنده می‌تواند با عامل دقیقاً یکسان، هم شهودی کد بزند و هم مهندسی عامل‌محور کند. آنچه تعیین می‌کند، این است که **چقدر آگاهانه مهار را پیکربندی و اعمال می‌کنید**.

نسخهٔ روزمرهٔ این مشاهده

وقتی عامل کاری را اشتباه انجام می‌دهد، اولین غریزه سرزنش مدل است. اما اغلب ریشهٔ خطا در ابزاری است که وجود ندارد، قاعده‌ای است که مبهم تعریف شده، گاردریلی که اصلاً تعریف نشده، یا کانتکست ویندوای پر از نویز است. **بیشتر خطاهای عامل، اگر صادقانه بررسی شوند، در واقع خطای پیکربندی‌اند.**

۷. نقش در حال تحول توسعه‌دهنده

دو حالت کاری که توسعه‌دهندگان به‌راحتی میان آن‌ها جابه‌جا می‌شوند:

رهبر ارکستر

هدایت مستقیم و هم‌زمان: در ویرایشگر نوشته‌شدن کد را می‌بینی، با پرامپت و اصلاح هدایتش می‌کنی و کنترل دقیقی بر هر تغییر داری.

هماهنگ‌کننده

واگذاری (handoff) ناهم‌زمان: هدف‌ها را تعریف می‌کنی، به عامل‌ها می‌سپاری و نتیجه را بازبینی می‌کنی. عامل‌ها در پس‌زمینه و موازی کار می‌کنند.

حالت رهبر ارکستر

این حالت برای منطق پیچیده، دیباگ دشوار یا کار در کدبیس ناآشنا مناسب است؛ جایی که باید هر تغییر را همان لحظه بفهمی. ابزارهایی مثل GitHub Copilot، Gemini Code Assist، Cursor و Windsurf عمدتاً همین حالت را پشتیبانی می‌کنند. این حالت برای مهندسانی که پیشینهٔ سنتی دارند طبیعی است و حس فهم و کنترل را حفظ می‌کند. **خطرش** این است که خود توسعه‌دهنده به گلوگاه تبدیل شود: اگر قرار باشد تک‌تک ضربه‌های کلید را خودت هدایت کنی، سرعت کار چندان بیشتر نمی‌شود.

حالت هماهنگ‌کننده

در این حالت، توسعه‌دهنده در سطح انتزاع بالاتری کار می‌کند و دیگر نوشته‌شدن کد را خط‌به‌خط دنبال نمی‌کند. این حالت برای کارهایی مناسب است که به‌خوبی تعریف شده‌اند: رفع اشکال، پیاده‌سازی ویژگی بر پایهٔ الگوهای موجود، مهاجرت کدبیس و تولید تست. ابزارهایی مثل Jules، حالت عاملی Copilot، عامل‌های پس‌زمینهٔ Cursor و Claude Code این حالت را با اجرای ناهم‌زمان کار، اغلب در محیط‌های سندباکسی با دسترسی کامل به ریپو و ابزار ساخت، پشتیبانی می‌کنند.

این حالت مهارت‌های متفاوتی می‌طلبد؛ به‌جای تسلط عمیق بر نحو و ریزه‌کاری‌های زبان، این مهارت‌ها لازم است:

- **مشخصات‌نویسی:** تعریف کار آن‌قدر دقیق که عامل بتواند بدون ابهام انجامش دهد.
- **تجزیه** `(decomposition)`**:** تقسیم کار بزرگ به بخش‌هایی که اندازه‌شان برای انجام‌دادن به‌دست عامل مناسب باشد.
- **ارزیابی:** سنجش سریع اینکه خروجی عامل به استانداردهای کیفیت می‌رسد یا نه.
- **طراحی سامانه:** طراحی قیدها، تست‌ها و حلقه‌های بازخوردی که کار عامل‌ها را بهره‌ور نگه می‌دارند.

مسئلهٔ ۸۰ درصد

یکی از چالش‌های همیشگی توسعهٔ مبتنی بر هوش مصنوعی: عامل‌ها می‌توانند حدود ۸۰٪ کد یک ویژگی را به‌سرعت تولید کنند، اما ۲۰٪ باقی‌مانده — یعنی حالت‌های مرزی، مدیریت خطا، نقاط اتصال به بخش‌های دیگر سامانه، و جزئیات حساسی که درستی کار به آن‌ها بستگی دارد — شناخت عمیقی از بستر پروژه می‌خواهد که مدل‌های امروز اغلب ندارند.

ماهیت خطاها هم تغییر کرده است: از اشتباه‌های سادهٔ نحوی به خطاهای مفهومیِ پنهان‌تر و خطرناک‌تر؛ مثل پیش‌فرض‌های غلط دربارهٔ منطق کسب‌وکار، نپرسیدن سؤال وقتی نیازمندی‌ها مبهم‌اند، نادیده‌گرفتن حالت‌های مرزی، و تصمیم‌های معماری‌ای که در درازمدت بار نگه‌داری پنهانی می‌سازند. تشخیص این خطاها دقیقاً به این دلیل سخت‌تر است که کد «درست به‌نظر می‌رسد» و حتی ممکن است تست‌های پایه را هم پاس کند.

> **رویکرد کارآمدترین توسعه‌دهندگان**
> هوش مصنوعی را برای کارهایی به کار می‌گیرند که در آن‌ها خوب است (پیاده‌سازی سریع کارهایی که به‌خوبی تعریف شده‌اند) و توجه خودشان را برای جاهایی نگه می‌دارند که هوش مصنوعی در آن‌ها ضعیف است (نیازمندی مبهم، موازنه‌های معماری، راستی‌آزمایی درستی کار). آن‌ها سرعت را از پذیرفتن همه‌چیز به دست نمی‌آورند؛ از متمرکزکردن تخصصشان روی جایی به دست می‌آورند که بیشترین اهمیت را دارد.

۸. عامل‌های کدنویسی در عمل

سه جایگاه در روز کاری

در ویرایشگر

تکمیل درون‌خطی، پنل گفت‌وگو برای توضیح یا تغییر کد همان‌جا، و آگاهی از کل کدبیس. جایی که بیشتر افراد نخستین بار با هوش مصنوعی آشنا می‌شوند و توسعه‌دهنده بی‌آنکه از ویرایشگر بیرون برود، تمرکز کارش را حفظ می‌کند.

در ترمینال

این عامل از خط فرمان اجرا می‌شود، هدف را به زبان ساده می‌گیرد و در کل کدبیس کار می‌کند: به کل فایل‌سیستم دسترسی دارد، چند فایل را با هم ویرایش می‌کند، ابزار و تست اجرا می‌کند و بر پایهٔ نتیجه، کار را دوباره تکرار می‌کند. امروز جدی‌ترین کدنویسی شهودی همین‌جا اتفاق می‌افتد.

در پس‌زمینه

این عامل کاری را تحویل می‌گیرد، به‌صورت خودگردان و گاهی چند ساعت در سندباکس ابری روی آن کار می‌کند و معمولاً در پایان یک پول‌ریکوئست تحویل می‌دهد. توسعه‌دهنده کار را می‌سپارد و بعداً نتیجه را بازبینی می‌کند.

در عمل، عامل ویرایشگر وقتی به کار می‌آید که وسط نوشتن کد هستید و بی‌آنکه تمرکزتان به هم بخورد، پیشنهاد یا توضیح سریع می‌خواهید. عامل ترمینال برای کارهای چندفایلی، کاوش کدبیس ناآشنا و کارهایی مناسب است که در آن‌ها عامل باید کد را اجرا کند و به نتیجه‌ای که می‌بیند واکنش نشان دهد. عامل پس‌زمینه برای کاری مناسب است که بتوانید در یک پاراگراف توصیفش کنید و سراغ کار دیگری بروید: رفع اشکالی شناخته‌شده، تولید مجموعه‌تست، یا مهاجرت کد از یک چارچوب به چارچوب دیگر. **انتخاب درست به ماهیت کار بستگی دارد، نه به اینکه کدام نوع عامل خودگردان‌تر است.**

وقتی خودِ محصول یک عامل است

هرچه تا اینجا گفتیم دربارهٔ به‌کارگیری عامل‌ها برای ساختن نرم‌افزار بود. اما اگر چیزی که می‌خواهید بسازید خودش یک عامل باشد، چه؟ مثلاً رباتی که درخواست‌های بازپرداخت را رسیدگی می‌کند، دستیاری پژوهشی که منابع را با هم مقایسه می‌کند و گزارش مستند می‌نویسد، یا ابزاری داخلی که رعایت مقررات را پایش می‌کند و موارد غیرعادی را گزارش می‌دهد. این‌ها محصول‌اند و ابزار، حافظه، ارزیابی و زیرساخت دیپلوی مخصوص خودشان را لازم دارند.

نکتهٔ تازه این است که همان گردش‌کار (workflow) مبتنی بر ترمینال که با آن اسکریپت نمونه ساخته می‌شد، حالا برای ساختن این عامل‌های تولیدی هم به کار می‌رود. برای نمونه، Agents CLI گوگل ابزار کوچکی است که مجموعه‌ای از مهارت‌ها را به عامل کدنویسیِ دلخواه شما اضافه می‌کند و کل چرخهٔ ساخت، ارزیابی، دیپلوی و رصد را پوشش می‌دهد:

```
# one-time setup
uvx google-agents-cli setup

# then, inside your coding agent:
> Build a support agent that answers questions from our docs.
> Evaluate it on the FAQ dataset.
> Deploy it to Agent Engine.
```

پشت همین چند جمله، عامل کدنویسی پروژه را از قالب می‌سازد، کد را می‌نویسد، مجموعهٔ ارزیابی تولید می‌کند، آن را اجرا می‌کند، روی رانتایم (runtime) عامل مستقر می‌کند و گزارش می‌دهد. توسعه‌دهنده SDK تازه‌ای یاد نمی‌گیرد؛ توصیف می‌کند و عامل از مهارت‌ها استفاده می‌کند.

هماهنگی میان چند عامل

برای حالت‌های ساده، وضعیت مشترک نشست کافی است؛ برای دسترسی به ابزار، MCP؛ و برای واگذاری کار میان عامل‌ها، پروتکل A2A. تیم مهندسی Anthropic در اوایل ۲۰۲۶ آزمایشی منتشر کرد که در آن تیم‌هایی از عامل‌ها روی چنین معماری‌ای، ظرف دو هفته یک کامپایلر زبان C را به زبان Rust ساختند؛ انسان‌ها جهت می‌دادند و خروجی را بازبینی می‌کردند اما پیاده‌سازی را نمی‌نوشتند. گلوگاه از نوشتن کد به **مشخص‌کردن اینکه چه باید بشود و راستی‌آزمایی اینکه شده** منتقل شد.

۹. هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی

بحث دربارهٔ اثر هوش مصنوعی معمولاً فقط حول «سرعت توسعه‌دهنده» می‌چرخد. اما برای مدیران فنی، معیار مهم‌تر **هزینهٔ کل مالکیت** است. برای فهم هزینهٔ واقعی باید دید هر گردش‌کار، بار مالی را چگونه میان CapEx (سرمایه‌گذاری اولیه برای ساختن) و OpEx (هزینهٔ جاری اجرا، اصلاح و نگه‌داری) تقسیم می‌کند. و در دوران هوش مصنوعی، OpEx را عمدتاً **اقتصاد توکن** تعیین می‌کند.

بدهی پنهان کدنویسی شهودی: سرمایه‌گذاری کم، هزینهٔ عملیاتی زیاد

در نگاه اول، کدنویسی شهودی فوق‌العاده مقرون‌به‌صرفه است: مانع ورود تقریباً صفر است — یک اشتراک ماهانه و چند پرامپت ساده. اما اقتصادش بار عملیاتی انباشته‌ای را پنهان می‌کند:

- **هدررفت توکن:** ریختن فایل‌های حجیم و بی‌ساختار در کانتکست ویندو (`context window`) و خواستن مکرر از مدل برای درست‌کردن اشتباه‌هایی که خودش ساخته و کسی بررسی‌شان نکرده، چرخه‌ای پرهزینه می‌سازد که توکن‌ها را هدر می‌دهد، چون کار به‌ندرت در دفعهٔ اول درست درمی‌آید.
- **هزینهٔ پنهان نگه‌داری:** کدی که با پرامپت‌های سرِدستی نوشته شده، اغلب ساختار منسجمی ندارد. شش ماه بعد که اشکالی پیدا شود، مهندسان باید روزها وقت بگذارند تا از این کد «اسپاگتی» سر دربیاورند.
- **هزینهٔ رفع مشکلات امنیتی:** بدون مهاری برای ارزیابی خودکار، تولید سریع کد یعنی تولید سریع آسیب‌پذیری. هزینهٔ رفع یک نقص امنیتی در محیط تولید، چندین برابر هزینهٔ پیدا کردنش در مرحلهٔ طراحی است.

سرمایه‌گذاری مهندسی عامل‌محور: سرمایه‌گذاری زیاد، هزینهٔ عملیاتی کم

مهندسی عامل‌محور این معادلهٔ اقتصادی را برعکس می‌کند. پیش از آنکه نخستین خط کد تولیدی نوشته شود، باید آگاهانه زمان مهندسی را سرمایه‌گذاری کرد: طراحی اسکیماهای `API`، ساختن مجموعه‌تست‌های قطعی و — از همه مهم‌تر — سازمان‌دادن به زمینهٔ عامل. هزینهٔ اولیهٔ این رویکرد بیشتر است، اما در عوض، هزینهٔ عرضه و نگه‌داری هر ویژگیِ تازه به‌شدت کاهش می‌یابد، چون عامل درون «کارخانه‌ای» کار می‌کند که با دقت حکمرانی می‌شود؛ پس خروجی‌اش از نظر ساختاری سالم است، از پیش تست شده و با استانداردهای سازمان هم‌راستاست.

مهندسی زمینه به‌عنوان اهرم مالی

در اقتصاد توکن، مهندسی زمینه فقط یک مهارت فنی نیست؛ یک راهبرد مالی است. مدل‌ها بابت هر تکه اطلاعاتی که می‌فرستید هزینه می‌گیرند و فرستادن ریپوی ۱۰۰,۰۰۰ توکنی در هر پرامپت، وقتی تعداد پرامپت‌ها زیاد شود، اقتصادی نیست. مهندسی زمینهٔ خوب تضمین می‌کند که مدل اطلاعاتی فشرده و پرسیگنال دریافت کند (مثل یک `AGENTS.md` دقیق و گاردریل‌های معماری)، نه انبوهی پر از نویز. وقتی زمینهٔ درست از همان ابتدا فراهم شود، احتمال درست‌شدن کار در دفعهٔ اول بالا می‌رود و چرخه‌های پرهزینهٔ آزمون‌وخطا از بین می‌روند.

مسیریابی هوشمند مدل

در کدنویسی شهودی، معمولاً برای هر کاری سراغ یکی از بزرگ‌ترین و پیشرفته‌ترین مدل‌ها می‌روند و گران‌ترین نرخ را می‌پردازند، حتی برای اصلاح یک غلط تایپی یا نوشتن یک تست واحد ساده. مدل کارخانه‌ای که خوب طراحی شده باشد جلوی این هدررفت را می‌گیرد: مدل‌های بزرگ برای کارهای بسیار پیچیده (نیازمندی، معماری، پیاده‌سازی اولیه) و مسیریابی خودکار کارهای قطعی‌تر و کم‌پیچیدگی‌تر (تولید تست، بازبینی کد، پایش `CI/CD`) به مدل‌های کوچک‌تر، سریع‌تر و بسیار ارزان‌تر.

۱۰. از کجا شروع کنیم

اصل راهنما

هوش مصنوعی هر فرهنگ مهندسی‌ای را که در آن به کار گرفته شود، **تقویت** می‌کند. توصیه‌های زیر، همین اصل را به عمل ترجمه می‌کنند.

برای تک‌تک توسعه‌دهندگان

1. **یک `AGENTS.md` (یا معادلش) برای پروژه بسازید.** قالبی را انتخاب کنید که عامل کدنویسی‌تان آن را می‌شناسد. با ده خط شروع کنید: پشتهٔ فناوری، قراردادهای کدنویسی، قواعد سخت و گردش‌کار. هر بار که عامل کاری کرد که نباید تکرار شود، یک قاعده به فایل اضافه کنید.
2. **مجموعه‌ای از مهارت‌ها را برای عامل کدنویسی‌تان نصب کنید** تا ساخت، ارزیابی، دیپلوی و بهینه‌سازی عامل‌ها را پوشش دهند.
3. **یک گردش‌کار تکراری را انتخاب و به نخستین عامل‌تان تبدیل کنید.** ساختن یک عامل از ابتدا تا انتها، بیشتر از خواندن دربارهٔ صد عامل به شما یاد می‌دهد.
4. **تست‌ها و ارزیابی‌ها را پیش از تولید کد بنویسید.** این دو با هم قرارداد شما با هوش مصنوعی‌اند و قصد را دقیق‌تر از هر پرامپت زبان طبیعی منتقل می‌کنند.
5. **هر خطی را که قرار است منتشر شود بازبینی کنید.** به هر کدی که بیش از حد «زیرکانه» به‌نظر می‌رسد شک کنید. بررسی کنید بسته‌های واردشده واقعاً وجود دارند. کدی که تیم آن را نمی‌فهمد، در آینده هزینهٔ دیباگی به بار می‌آورد که تیم از پسش برنمی‌آید.
6. **مهارت‌های بنیادی خود را زنده نگه دارید.** هوش مصنوعی کارهای روزمره را انجام می‌دهد تا شما روی کارهای چالش‌برانگیز تمرکز کنید؛ اما این تقسیم کار فقط وقتی جواب می‌دهد که مهارت دیباگ، طراحی سامانه و درک شهودی‌تان از کارایی و درستی همچنان قوی بماند.

برای مدیران فنی

1. **مهندسی زمینه را به یکی از رویه‌های اصلی تیم تبدیل کنید.** با `AGENTS.md`، سیستم پرامپت‌ها، مجموعه‌های ارزیابی و کتابخانه‌های مهارت مثل کد رفتار کنید: در پول‌ریکوئست بازبینی شوند، همراه پروژه نسخه‌بندی شوند و مالک مشخصی داشته باشند. بدون این انضباط، مهار منحرف می‌شود و رفتار عامل در تیم تکرارناپذیر می‌گردد.
2. **معیار پذیرش را ارزیابی بگذارید، نه نمایش.** یک نمایش موفق فقط ثابت می‌کند عامل «یک‌بار» می‌تواند؛ مجموعهٔ ارزیابی‌ای که پاس شده ثابت می‌کند «همیشه» می‌تواند. اما ارزیابی‌ای که معیار روشنی نداشته باشد،

در واقع چیزی را نمی‌سنجد: مشخص کنید چه چیزی را نمره می‌دهید — موفقیت کار، کیفیت استفاده از ابزار، انطباق مسیر، توهم، و کیفیت پاسخ.

3. **فرایند بازبینی کد را متناسب با کدی که هوش مصنوعی تولید می‌کند از نو طراحی کنید.** کد تولیدشده به همان اندازه یا بیشتر موشکافی می‌خواهد، با توجه ویژه به وابستگی‌های توهمی، مدیریت خطای ناکافی و ایرادهای پنهان در درستی کار که در نگاه اول درست به‌نظر می‌رسند.

4. **در قواعد تیم، مرز کار نمونه‌سازی و کار تولیدی را روشن کنید.** کدنویسی شهودی سرعت مناسب برای کاوش را می‌دهد و مهندسی عامل‌محور انضباط لازم برای تولید را. تیم‌هایی که این مرز را مبهم می‌گذارند، در نهایت نمونه‌های اولیه‌ای را ناخواسته منتشر می‌کنند.

5. **روی اجزای مهار به‌عنوان دارایی مشترک تیم سرمایه‌گذاری کنید.** سیستم پرامپت‌های قابل استفادهٔ مجدد، کتابخانه‌های مهارت، اتصال‌های سرور `MCP` و مهارهای ارزیابی، ارزش‌شان در پروژه‌ها انباشته می‌شود. با آن‌ها مثل زیرساخت رفتار کنید.

برای سازمان‌ها

1. **توسعهٔ مبتنی بر هوش مصنوعی را یک سرمایه‌گذاری مهندسی ببینید، نه ابزاری برای افزایش بهره‌وری.** اگر عامل کدنویسی را بدون زیرساخت ارزیابی، رصدپذیری و استاندارد معماری به کار بگیرید، سرعت به دست می‌آورید اما کیفیت را از دست می‌دهید و بدهی فنی سریع‌تر از توان تیم برای تسویه‌اش انباشته می‌شود.

2. **پیش از گسترش استفاده، زیرساخت تولید را بسازید.** ارزیابی مسیر و پاسخ نهایی در `CI`، ردیابی هر اجرای عامل، مجوزهای محدودشده به ازای هر عامل، و بازبینی امنیتی تنظیم‌شده برای حالت‌های شکست کد تولیدشده — این بستر باید پیش از انتشار نخستین عامل تولیدی آماده باشد، نه بعد از آن.

3. **برای ابزارها و ارتباط میان عامل‌ها، استانداردهای باز را انتخاب کنید.** `MCP` برای دسترسی به ابزار و `A2A` برای واگذاری میان عامل‌ها در حال تبدیل‌شدن به اسکلت ارتباطی سامانه‌های چندعاملی‌اند.

4. **برای تیم‌هایی برنامه‌ریزی کنید که انسان و عامل را کنار هم دارند.** قوی‌ترین نتایج تولیدی از معماری‌هایی آمده که انسان جهت می‌دهد، عامل پیاده‌سازی می‌کند و پروتکل‌های روشنِ واگذاری کار، مرز میان آن دو را مشخص می‌کنند.

5. **در استخدام و آموزش، توان قضاوت را محور قرار دهید.** با خودکارتر شدن پیاده‌سازی، گلوگاه به مشخصات‌نویسی، ارزیابی، قضاوت معماری و بازبینی منتقل می‌شود.

۱۱. جمع‌بندی و سه اصل ماندگار

گذار از نحو به قصد، پیش‌بینی آینده نیست؛ واقعیت امروز است. توسعه‌دهندگان همین حالا هم بیشتر وقتشان را صرف توصیف «چه می‌خواهند» می‌کنند تا «چگونه ساخته شود». پرسش این نیست که آیا این دگرگونی رخ می‌دهد یا نه؛ پرسش این است که افراد، تیم‌ها و سازمان‌ها تا چه اندازه موفق از آن عبور خواهند کرد.

۱. در مقیاس بزرگ، ساختار جواب می‌دهد، نه حس‌وحال

کدنویسی شهودی برای کاوش، نمونه‌سازی و پروژهٔ شخصی رویکرد درستی است. اما برای نرم‌افزاری که سازمان به آن تکیه می‌کند، انضباط مهندسی عامل‌محور یک انتخاب نیست، یک ضرورت است. قطعی سرویس، آسیب‌پذیری امنیتی و دردسرهای بزرگ نگه‌داری درست در فاصلهٔ «به‌نظر کار می‌کند» تا «در همهٔ شرایط درست کار می‌کند» پیدا می‌شوند.

۲. هوش مصنوعی فرهنگ مهندسی را تقویت می‌کند

سازمان‌هایی که رویه‌های تست قوی، استانداردهای معماری روشن و بازبینی کد سالم دارند، سود بسیار بیشتری از توسعهٔ مبتنی بر هوش مصنوعی می‌برند. هوش مصنوعی مثل یک اهرم عمل می‌کند و هم نقاط قوت و هم نقاط ضعف را چند برابر می‌کند.

۳. نقش انسان کوچک نمی‌شود، بلکه تغییر می‌کند

توسعه‌دهندگانی که معماری را می‌فهمند، می‌توانند مشخصات دقیق بنویسند، خروجی را نقادانه بسنجند و سامانه‌های مؤثری از قیدها و حلقه‌های بازخورد طراحی کنند، امروز بیش از هر زمان دیگری ارزشمندند.

> تولید حل شده است. راستی‌آزمایی، قضاوت و جهت‌دهی، پیشهٔ تازه‌اند.

۱۲. خودآزمایی

پیش از جلسهٔ بعد، پاسخ این پرسش‌ها را برای خودتان بنویسید. اگر برای یکی پاسخ روشنی ندارید، به بخش مربوط برگردید.

1. تفاوت ربات گفت‌وگو و عامل را در یک جمله بیان کنید. کدام جزء از پنج جزء، این تفاوت را می‌سازد؟
2. چرا «استفاده از هوش مصنوعی» معیار خوبی برای تشخیص کدنویسی شهودی از مهندسی عامل‌محور نیست؟ معیار درست چیست؟
3. یک مثال بزنید که در آن تست کافی است و ارزیابی لازم نیست؛ و یک مثال که عکس آن صادق است.
4. سه چیز را نام ببرید که در پروژهٔ فعلی شما باید در زمینهٔ ایستا باشند و سه چیز که باید پویا بمانند. چرا؟

5. معادلهٔ «عامل = مدل + مهار» چه پیامد عملی‌ای برای بودجهٔ تیم شما دارد؟
6. آخرین باری که عامل اشتباهی کرد، کدام جزء مهار غایب بود؟
7. در کدام کارها شما «رهبر ارکستر (Conductor)» هستید و در کدام‌ها «هماهنگ‌کننده (Orchestrator)»؟ آیا این انتخاب آگاهانه است؟
8. در مسئلهٔ ۸۰ درصد، ۲۰٪ باقی‌ماندهٔ آخرین ویژگی‌ای که ساختید چه بود؟
9. اگر بخواهید هزینهٔ توکن تیم را نصف کنید بی‌آنکه کیفیت افت کند، از کجا شروع می‌کنید؟
10. یک قاعدهٔ سخت بنویسید که امروز باید در `AGENTS.md` پروژه‌تان باشد اما نیست.

منبع: این جزوه خلاصه و بازنویسی آموزشی مقالهٔ The New SDLC With Vibe Coding: From ad-hoc prompting to Agentic Engineering نوشتهٔ آدی عثمانی، شوبهام سابو و سوکراتیس کارتاکیس است (مجموعهٔ مقالات عامل‌ها، می ۲۰۲۶). ترجمهٔ کامل فارسی مقاله در همین پوشه در دسترس است.

واژه‌نامهٔ این جلسه

اصطلاحاتی که در این سند به کار رفته‌اند، به‌همراه معادل انگلیسی‌شان. فهرست کامل 159 اصطلاح دوره در واژه‌نامهٔ دوره آمده است.

اصطلاح	معادل انگلیسی	توضیح
ابزار	`tool`	تابعی که عامل می‌تواند فرا بخواند تا در جهان بیرون کنش کند.
ارزیابی	`evaluation`	سنجش رفتار غیرقطعی با نمره و باند تحمل، در برابر ادعای صفر و یکی تست.
ارکستراسیون	`orchestration`	هماهنگ‌کردن چند عامل یا چند گام اجرا.
افشای تدریجی	`Progressive disclosure`	بارگذاری سه‌سطحی: فراداده همیشه، بدنه هنگام فعال‌شدن، منابع همراه فقط در صورت نیاز.
انحراف	`drift`	دورشدن تدریجی رفتار از آنچه انتظار می‌رفت.
بازیابی	`retrieval`	آوردن دانش بیرونی به زمینه در لحظهٔ نیاز.
بنچمارک	`benchmark`	مجموعه‌کار استاندارد برای مقایسهٔ عملکرد.
تأخیر	`latency`	زمان سپری‌شده تا دریافت پاسخ.
تجزیه	`decomposition`	شکستن کاری بزرگ به زیرکارهای اتمی.
تست	`test`	وارسی قطعی رفتار کد. «یونیت تست» = آزمون واحد.
توکن	`token`	واحد کوچکی از متن که مدل پردازش می‌کند.
حافظه	`memory`	آنچه عامل میان نوبت‌ها یا نشست‌ها نگه می‌دارد.
حافظهٔ رویه‌ای	`Procedural memory`	به‌خاطرسپردن «چگونه انجام دادن»؛ در برابر حافظهٔ رویدادی (چه گذشت) و معنایی (چه می‌دانیم).
حلقه	`loop`	چرخهٔ ادراک ← برنامه ← کنش ← مشاهده که عامل در آن می‌چرخد.
خودگردان	`autonomous`	توان اجرای کار بدون تأیید انسانی در هر گام.

اصطلاح	معادل انگلیسی	توضیح
داور	`judge`	مدلی که خروجی مدل دیگر را در برابر سنجه نمره می‌دهد.
دروازه	`gate`	نقطهٔ وارسی که اجرا را تا برآورده‌شدن شرطی متوقف می‌کند.
دستور	`instruction`	قاعده‌ای که رفتار عامل را شکل می‌دهد، در برابر داده‌ای که پردازش می‌کند.
دیباگ	`debug`	یافتن و رفع علت ریشه‌ای خطا.
دیپلوی	`deploy`	انتشار کد روی محیط اجرا.
رانتایم	`runtime`	محیطی که عامل یا برنامه در آن اجرا می‌شود.
رصدپذیری	`observability`	توان دیدن آنچه درون سامانه رخ می‌دهد.
رهبر ارکستر	`Conductor`	کار هم‌زمان و دستی با عامل در ویرایشگر؛ کنترل دقیق بر هر تغییر.
ریپو	`repository`	مخزن کد تحت کنترل نسخه.
زمینه	`context`	اطلاعاتی که عامل در لحظه در اختیار دارد. پایهٔ «مهندسی زمینه» و «پوسیدگی زمینه».
زمینهٔ ایستا / پویا	`Static`	ایستا همیشه بارگذاری می‌شود (هویت عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی سند).
زیرعامل	`subagent`	عاملی که عامل دیگری برای کاری مشخص فرا می‌خواند.
سنجه	`rubric`	معیاری که بر پایه‌اش به خروجی نمره داده می‌شود.
سندباکس	`sandbox`	محیط اجرای ایزوله و کم‌امتیاز برای مهار شعاع انفجاری.

اصطلاح	معادل انگلیسی	توضیح
عامل	`agent`	مدل به‌علاوهٔ مهار؛ توان استدلال، به‌کارگیری ابزار و اجرای کار.
فراخوان ابزار	`tool call`	یک نوبت مشخص از به‌کارگیری ابزار، با آرگومان‌ها و نتیجه‌اش.
قصد / نیت	`intent`	خواستهٔ واقعی کاربر، در برابر آنچه لفظاً گفته. ریشهٔ «انحراف قصد» و «رضایت قصد».
قطعی / غیرقطعی	`deterministic`	قطعی یعنی ورودی یکسان همیشه خروجی یکسان می‌دهد؛ مدل‌ها غیرقطعی‌اند.
قلاب	`hook`	کد قطعی که در نقطهٔ مشخصی از چرخهٔ عمر اجرا می‌شود.
قید	`constraint`	محدودیت سختی که فضای کنش عامل را کوچک می‌کند.
مدل کارخانه	`Factory model`	محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه سامانه‌ای است که کد را تولید می‌کند.
مرز	`boundary`	خط جدایی میان دو ناحیهٔ اعتماد یا مسئولیت.
مسیر	`trajectory`	توالی گام‌های استدلال و فراخوان ابزار که عامل تا رسیدن به پاسخ طی می‌کند.
مسیریابی	`routing`	انتخاب اینکه کدام مهارت یا عامل کار را بردارد.
مشخصات	`spec`	سند قصد و رفتار مورد انتظار؛ منبع حقیقت برای عامل و انسان.
مهار	`harness`	هرچه پیرامون مدل قرار می‌گیرد: پرامپت، ابزار، سندباکس، ارکستراسیون، قلاب. عامل = مدل + مهار.

اصطلاح	معادل انگلیسی	توضیح
مهارت	`skill`	پوشهٔ دانش رویه‌ای که عامل فقط هنگام نیاز بارگذاری می‌کند.
مهندسی زمینه	`Context engineering`	طراحی آگاهانهٔ اطلاعاتی که عامل در اختیار دارد: دستورها، دانش، حافظه، مثال‌ها، ابزارها و گاردریل‌ها.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها با نظارت انسانی بر معماری.
نشست	`session`	کل یک گفت‌وگوی چندنوبتی، از آغاز تا پایان کار.
هماهنگ‌کننده	`Orchestrator`	واگذاری ناهم‌زمان کار به چند عامل و بازبینی نتیجه؛ کار در سطح انتزاع بالاتر.
وابستگی	`dependency`	کتابخانهٔ بیرونی که پروژه به آن تکیه دارد.
واگذاری	`handoff`	سپردن کار از عاملی به عامل دیگر.
پرامپت	`prompt`	متنی که به مدل داده می‌شود. پیش‌تر «درخواست» بود که با request اشتباه می‌شد.
پوسیدگی زمینه	`Context rot`	افت کیفیت مدل با بزرگ‌شدن ورودی، حتی وقتی دشواری کار ثابت است.
پوشش	`coverage`	سهمی از رفتار که آزمون یا ارزیابی واقعاً می‌سنجد.
پول‌ریکوئست	`pull request`	درخواست ادغام تغییرها در شاخهٔ اصلی.
کامیت	`commit`	ثبت یک تغییر در تاریخچهٔ گیت.
کانتکست ویندو	`context window`	بیشینهٔ توکنی که مدل در یک نوبت می‌تواند ببیند.

اصطلاح	معادل انگلیسی	توضیح
کد دورریختنی	`Disposable code`	وقتی مشخصات محکم باشد، کل کدبیس بارها بازتولید می‌شود؛ پس دلبستگی عاطفی به کد معنا ندارد.
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
گاردریل	`guardrail`	قید بیرونی که کنش‌های مجاز عامل را محدود می‌کند.
گردش‌کار	`workflow`	توالی گام‌های یک کار تکرارشونده.
